

I vai criar um arquivo completo em Markdown (.md) contendo o resumo do conteúdo das 5 aulas, exemplos explicativos de cada tópico e uma seção de exercícios com testes de mesa e resoluções detalhadas passo a passo.

```
# Guia de Revisão: Análise e Projeto de Algoritmos
## Depuração, Teste de Mesa e Documentação de Código
```

1. Fundamentos de Algoritmos

1.1 O que é um Algoritmo?

Um **algoritmo** é uma sequência finita e bem definida de passos lógicos para resolver um problema.

1.2 Estruturas Básicas

Todo algoritmo é construído combinando três estruturas fundamentais:

1. **Sequência:** Execução de instruções passo a passo, de cima para baixo.
2. **Seleção (Condicionais):** Tomada de decisões baseada em condições booleanas (**se / senão**, **if / else**).
3. **Repetição (Laços / Loops):** Repetição de um bloco de código enquanto ou para determinada condição.

2. Representação e Teste de Mesa

2.1 O que é um Teste de Mesa?

O **Teste de Mesa** (ou *dry run*) é uma técnica manual para simular a execução de um algoritmo.

Como montar uma Tabela de Teste de Mesa:

- Crie colunas para a **Instrução/Linha**, as **Variáveis** do programa e a **Saída/Impressão**.
- Atualize os valores das variáveis conforme as instruções são processadas linha por linha.

3. Tipos de Erros de Software

Tipo de Erro	Quando Ocorre?	Características	Exemplo
Erro de Sintaxe	Antes/Durante a Compilação	Violação das regras gramaticais da linguagem.	
Erro de Execução (Runtime)	Durante a Execução	O código compila, mas sofre um colapso e l	
Erro de Lógica	Durante a Execução	O programa roda até o fim sem quebrar, porém gera um r	

4. Depuração (Debugging) com IDE (IntelliJ)

4.1 Conceitos e Ferramentas

- * **Modo Debug:** Modo de execução controlado pela IDE (ícone do besouro/bug) que permite pausar e
- * **Breakpoint (Ponto de Interrupção):** Marcação em uma linha de código onde a execução do programa
- * **Conditional Breakpoint:** Breakpoint acionado **somente** se uma condição booleana for verdadeira
- * **Evaluate Expression (Alt + F8):** Permite executar expressões de código dinamicamente no console
- * **Rubber Duck Debugging:** Técnica de explicar a lógica do código em voz alta (para um colega ou

4.2 Comandos de Navegação do Depurador

[Linha Atual]	Step Over (F8)	Pula para a próxima linha no mesmo método	Step Into (F7)
	Entra no método chamado na linha atual	Step Out (Shift+F8)	Executa o resto do método atual e volta para quem chamou

```
* **Step Over (`F8`):** Executa a linha atual por inteiro. Se houver uma chamada de método, execut
* **Step Into (`F7`):** Se a linha atual tiver uma chamada de método, entra dentro desse método pa
* **Step Out (`Shift + F8`):** Conclui a execução do método atual e retorna para o chamador.
```

5. Documentação de Código e Javadoc

5.1 Por que Documentar?

- * Previne esquecimento da própria lógica no futuro.
- * Evita retrabalho e reduz bugs na equipe.
- * Facilita o *onboarding* de novos desenvolvedores.
- * Melhora o entendimento por ferramentas de IA e de análise estática.

5.2 Documentação Útil vs. Inútil

```
* **Inútil:** Comentar o óbvio ou traduzir o código para português.
  * *Exemplo inútil:* `i++; // adiciona 1 no i`
* **Útil:** Explicar a **regra de negócio**, o **porquê** de uma decisão e os casos de borda.
  * *Exemplo útil:* `// Aplica 20% de desconto para locações acima de 10 dias conforme regra comer
```

5.3 Padrão Javadoc (Java)

Formato oficial de comentários do Java iniciado por `/\*\*` e finalizado por `\*/`.

```
```java
/**
 * Calcula o valor total do aluguel de um veículo aplicando descontos progressivos.
 *
 * @param valorDiaria Valor cobrado por dia de aluguel (deve ser positivo).
 * @param diasQuantidade Número total de dias de locação.
 * @return O valor total final com desconto aplicado se elegível.
 * @throws IllegalArgumentException se valorDiaria ou diasQuantidade forem menores ou iguais a zero
 */
public double calcularAluguel(double valorDiaria, int diasQuantidade) {
 if (valorDiaria <= 0 || diasQuantidade <= 0) {
 throw new IllegalArgumentException("Valores devem ser maiores que zero.");
 }
 double total = valorDiaria * diasQuantidade;
 if (diasQuantidade >= 10) {
 total *= 0.8; // 20% de desconto
 }
 return total;
}
```

- Geração Automática no IntelliJ: Menu Tools > Generate JavaDoc....

## 6. Exercícios de Revisão com Cálculos e Resoluções

### Exercício 1: Teste de Mesa - Estrutura Condicional

Dado o pseudocódigo abaixo, faça o teste de mesa e determine o valor final das variáveis a, b, c e d.

```

a, b, c, d : inteiro
a ← 1
b ← 3
se (a + b >= 5) então
 c ← 10
senão
 c ← 30
fimse
d ← a + b
a ← c
b ← d
escreva(a, b, c, d)

```

Resolução Passo a Passo (Tabela do Teste de Mesa):

Passo	Linha / Instrução	a	b	c	d	Condição (a + b >= 5) 5)	Observação
1	a ← 1	1	-	-	-	-	Inicialização de a
2	b ← 3	1	3	-	-	-	Inicialização de b
3	se (a + b >= 5)	1	3	-	-	1 + 3 = 4 >= 5 (Falso)	Desvia para o bloco <b>senão</b>
4	c ← 30	1	3	30	-	-	Atribuição no <b>senão</b>
5	d ← a + b	1	3	30	4	-	d = 1 + 3 = 4
6	a ← c	30	3	30	4	-	a recebe o valor de c (30)
7	b ← d	30	4	30	4	-	b recebe o valor de d (4)
8	escreva(...)	30	4	30	4	-	Impressão final

Resultado impresso: 30, 4, 30, 4

---

### Exercício 2: Teste de Mesa - Laço Simples (para)

Efetue o teste de mesa do pseudocódigo abaixo e determine a saída impressa no final:

```
total ← 0
para (i ← 0; i < 3; i++):
 total ← total + i + 1
imprimir(total)
```

#### Resolução Passo a Passo (Tabela do Teste de Mesa):

Passo	Iteração	i	i < 3	Cálculo total + i + 1	total (Novo Valor)
1	Inicial	-	-	-	0
2	1ª iteração	0	0 < 3 ( <b>V</b> )	0 + 0 + 1 = 1	<b>1</b>
3	2ª iteração	1	1 < 3 ( <b>V</b> )	1 + 1 + 1 = 3	<b>3</b>
4	3ª iteração	2	2 < 3 ( <b>V</b> )	3 + 2 + 1 = 6	<b>6</b>
5	Fim do laço	3	3 < 3 ( <b>F</b> )	Loop encerra	<b>6</b>

Resultado impresso: 6

---

### Exercício 3: Teste de Mesa - Laço com Condicional e Módulo (mod)

Acompanhe a execução do pseudocódigo abaixo e calcule o valor final de `total`:

```
total ← 0
para (i ← 1; i <= 5; i++):
 se (i mod 2 == 0) então
 total ← total + i
imprimir(total)
```

#### Resolução Passo a Passo:

Passo	i	i <= 5	i mod 2 == 0 (Par?)	Ação	total
1	-	-	-	Inicialização	0
2	1	Verdadeiro	1 mod 2 = 1 (Falso)	Não soma nada	0
3	2	Verdadeiro	2 mod 2 = 0 (Verdadeiro)	total = 0 + 2	<b>2</b>
4	3	Verdadeiro	3 mod 2 = 1 (Falso)	Não soma nada	2
5	4	Verdadeiro	4 mod 2 = 0 (Verdadeiro)	total = 2 + 4	<b>6</b>
6	5	Verdadeiro	5 mod 2 = 1 (Falso)	Não soma nada	6
7	6	Falso	-	Encerra laço	6

**Resultado impresso:** 6 (*Soma dos pares 2 + 4*)

---

#### Exercício 4: Teste de Mesa - Laços Aninhados (Nested Loops)

Realize o teste de mesa completo do algoritmo a seguir com laços encadeados:

```
total ← 0
para (i ← 1; i ≤ 3; i++):
 para (j ← 1; j ≤ i; j++):
 total ← total + j
imprimir(total)
```

#### Resolução Passo a Passo Detalhada:

Passo	i	i ≤ 3	j	j ≤ i	Operação (total + j)	total (Atualizado)
0	-	-	-	-	Estado inicial	0
1	1	Sim	1	1 ≤ 1 ( <b>V</b> )	0 + 1	1
2	1	Sim	2	2 ≤ 1 ( <b>F</b> )	Sai do laço interno	1
3	2	Sim	1	1 ≤ 2 ( <b>V</b> )	1 + 1	2
4	2	Sim	2	2 ≤ 2 ( <b>V</b> )	2 + 2	4
5	2	Sim	3	3 ≤ 2 ( <b>F</b> )	Sai do laço interno	4
6	3	Sim	1	1 ≤ 3 ( <b>V</b> )	4 + 1	5
7	3	Sim	2	2 ≤ 3 ( <b>V</b> )	5 + 2	7
8	3	Sim	3	3 ≤ 3 ( <b>V</b> )	7 + 3	10
9	3	Sim	4	4 ≤ 3 ( <b>F</b> )	Sai do laço interno	10
10	4	4 ≤ 3 ( <b>F</b> )	-	-	Encerra laço externo	10

**Resultado impresso:** 10

---

#### Exercício 5: Algoritmo Iterativo de Fibonacci

Dado o algoritmo abaixo para calcular o  $n$ -ésimo termo da sequência de Fibonacci:

```
algoritmo fibonacci(n):
 se n ≤ 1 então
 retorne n
 a ← 0
 b ← 1
 para i de 2 até n:
 c ← a + b
 a ← b
 b ← c
 retorne b
```

Calcule o valor retornado para `fibonacci(5)` mostrando o rastreamento de variáveis.

### Resolução Passo a Passo:

1. Entrada: `n = 5`
2. Teste inicial: `5 <= 1` é **Falso**.
3. Inicialização: `a = 0, b = 1`
4. Execução do laço para `i` de 2 até 5:

Iteração (i)	Cálculo <code>c = a + b</code>	Novo a ( <code>a = b</code> )	Novo b ( <code>b = c</code> )
<code>i = 2</code>	<code>c = 0 + 1 = 1</code>	<code>a = 1</code>	<code>b = 1</code>
<code>i = 3</code>	<code>c = 1 + 1 = 2</code>	<code>a = 1</code>	<code>b = 2</code>
<code>i = 4</code>	<code>c = 1 + 2 = 3</code>	<code>a = 2</code>	<code>b = 3</code>
<code>i = 5</code>	<code>c = 2 + 3 = 5</code>	<code>a = 3</code>	<code>b = 5</code>

**Resultado retornado:** 5

(Sequência de Fibonacci: 0, 1, 1, 2, 3, 5)

### Exercício 6: Classificação de Erros

Analise os 3 trechos de código Java a seguir e classifique o tipo de erro presente em cada um (Sintaxe, Execução ou Lógica):

#### Trecho A:

```
public boolean verificarDisponibilidade(Carro carro {
 if (carro.isDisponivel()) {
 return true
 }
 return false;
}
```

- **Tipo de Erro:** Erro de Sintaxe
- **Justificativa:** Faltam parênteses de fechamento nos parâmetros do método (`Carro carro` e falta ponto e vírgula ; após `return true`. O código nem chega a compilar.

#### Trecho B:

```
public int buscarIdadeDoCliente(String nome) {
 Cliente encontrado = null;
 for (Cliente c : clientes) {
 if (c.getNome().equals(nome)) {
 encontrado = c;
 }
 }
 return encontrado.getIdade();
}
```

- **Tipo de Erro:** Erro de Execução (Runtime Error)

- **Justificativa:** Se o cliente não for localizado na lista `clientes`, a variável `encontrado` continuará valendo `null`. Na última linha, ao tentar executar `encontrado.getIdade()`, o programa lançará uma exceção `java.lang.NullPointerException` e quebrará.
- 

#### Trecho C:

```
public double calcularDescontoEspecial(double valorTotal, int dias) {
 // Regra: Conceder 20% de desconto para locações de 10 dias ou mais
 if (dias < 10) {
 valorTotal = valorTotal * 0.8;
 }
 return valorTotal;
}
```

- **Tipo de Erro: Erro de Lógica**
  - **Justificativa:** O programa compila e roda sem erros, mas aplica o desconto para quem aluga por *menos* de 10 dias (`dias < 10`), em vez de aluguéis de 10 dias ou mais (`dias >= 10`). O resultado financeiro estará incorreto.
- 

#### Exercício 7: Identificação do Comando de Depuração

Suponha que você está depurando a seguinte classe no IntelliJ e o breakpoint parou na **linha 3**:

```
1: public void processarPedido(Pedido p) {
2: double total = p.calcularTotal();
3: enviarEmailConfirmacao(p.getCliente());
4: System.out.println("Pedido processado!");
5: }
```

Responda qual comando do depurador você deve acionar se desejar:

1. Ir para a linha 4 sem entrar no detalhamento do método `enviarEmailConfirmacao`.
2. Entrar no código do método `enviarEmailConfirmacao` para inspecionar como o e-mail é montado.
3. Se estiver dentro do método `enviarEmailConfirmacao` e desejar voltar imediatamente para a linha 4 de `processarPedido`.

#### Resolução:

1. **Step Over (F8):** Executa o método da linha 3 diretamente e avança para a linha 4.
  2. **Step Into (F7):** Entra na execução interna do método `enviarEmailConfirmacao`.
  3. **Step Out (Shift + F8):** Finaliza a execução do método atual e retorna ao chamador na linha seguinte.
- 

#### Exercício 8: Refatoração e Documentação Javadoc

Considere o método abaixo com comentários inadequados:

```
// Método que faz o cálculo
public double calc(double p, double t) {
 // calcula
```

```

 double r = p * (1 + t);
 // retorna
 return r;
}

```

Refatore o método alterando os nomes das variáveis para torná-las legíveis e adicione a documentação Javadoc completa e adequada.

### Resolução:

```

/**
 * Calcula o valor final com acréscimo de taxa percentual.
 *
 * @param valorPrincipal O valor base inicial sobre o qual a taxa será aplicada.
 * @param taxaPercentual A taxa a ser somada, expressa em valor decimal (ex: 0.10 para 10%).
 * @return O valor total atualizado acrescido da taxa.
 */
public double calcularValorComTaxa(double valorPrincipal, double taxaPercentual) {
 double valorComTaxa = valorPrincipal * (1 + taxaPercentual);
 return valorComTaxa;
}

```

## 7. Checklist de Estudo para a Prova

- ☐ Consigo preencher uma tabela de teste de mesa para laços para e enquanto?
- ☐ Sei a diferença entre Erro de Sintaxe, Execução e Lógica?
- ☐ Sei qual tecla atalho utilizar no IntelliJ para **Step Over** (F8), **Step Into** (F7) e **Step Out** (Shift+F8)?
- ☐ Sei para que serve um **Conditional Breakpoint** e a função **Evaluate Expression**?
- ☐ Entendo o conceito de **Rubber Duck Debugging**?
- ☐ Sei escrever tags Javadoc **@param**, **@return** e **@throws** corretamente?

O arquivo `revisao_algoritmos_depuracao.md` foi gerado com sucesso com o resumo das aulas, exemplos e exercícios com cálculos e resoluções completas.

Gostaria que eu transformasse este material em uma apresentação de slides HTML ou precisa de ajustes nos exercícios?# Material de Revisão: Análise e Projeto de Algoritmos (Depuração e Documentação)

## Módulo 1: Fundamentos e Representação de Algoritmos

Um **algoritmo** é uma sequência lógica e finita de instruções bem definidas para resolver um problema ou realizar uma tarefa. O fluxo de trabalho do desenvolvedor envolve entender o problema, escrever o código, testar, depurar, documentar e revisar.

### Estruturas Básicas de um Algoritmo

Todo algoritmo baseia-se em três estruturas fundamentais:

1. **Sequência:** Execução de instruções passo a passo de forma linear.

2. **Seleção (Condicionais):** Tomada de decisões com base em condições booleanas (**se** / **senão** ou **if** / **else**).
3. **Repetição (Laços/Loops):** Execução repetida de um bloco de código (**para**, **enquanto** ou **for**, **while**).

### Formas de Representação

- **Pseudocódigo:** Descrição estruturada do algoritmo em linguagem próxima à humana.
- **Fluxograma:** Representação gráfica do fluxo de execução utilizando formas geométricas padronizadas.

## Módulo 2: Teste de Mesa (Exercícios Resolvidos Passo a Passo)

O **teste de mesa** consiste em simular manualmente a execução de um algoritmo rastreando o valor de cada variável linha por linha em uma tabela.

### Exercício 1: Condicional e Reatribuição de Variáveis

Dado o pseudocódigo abaixo:

```

inicio
 a, b, c, d : inteiro;
 a ← 1;
 b ← 3;
 se (a + b >= 5) então:
 c ← 10;
 senão:
 c ← 30;
 fimse;
 d ← a + b;
 a ← c;
 b ← d;
 escreva(a, b, c, d);
fim

```

### Resolução e Tabela de Rastreio:

Passo	Instrução Executada	a	b	c	d	Avaliação da Condição
1	a ← 1; b ← 3;	1	3	-	-	-
2	se (a + b >= 5)	1	3	-	-	1 + 3 = 4 ≥ 5 (Falso)
3	Executa o bloco senão: c ← 30;	1	3	30	-	-
4	d ← a + b;	1	3	30	4	1 + 3 = 4
5	a ← c;	30	3	30	4	-

Passo	Instrução Executada	a	b	c	d	Avaliação da Condição
6	$b \leftarrow d;$	30	4	30	4	-
7	escreva(a, b, c, d)	30	4	30	4	<b>Saída:</b> 30, 4, 30, 4

## Exercício 2: Laço de Repetição Acumulador (para)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 0; i < 3; i++):
 total ← total + i + 1
imprimir(total)
```

### Resolução e Tabela de Rastreio:

Iteração	Valor de i	Condição (i < 3)	Expressão (total + i + 1)	Novo total
<b>Início</b>	-	-	-	0
<b>1ª</b>	0	0 < 3 (Verdadeiro)	0 + 0 + 1 = 1	1
<b>2ª</b>	1	1 < 3 (Verdadeiro)	1 + 1 + 1 = 3	3
<b>3ª</b>	2	2 < 3 (Verdadeiro)	3 + 2 + 1 = 6	6
<b>Fim</b>	3	3 < 3 (Falso)	Laço encerrado	<b>Saída:</b> 6

## Exercício 3: Laço com Filtro Condicional (mod)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 1; i <= 5; i++):
 se (i mod 2 == 0) então
 total ← total + i
imprimir(total)
```

### Resolução e Tabela de Rastreio:

Valor de i	Condição (i <= 5)	i mod 2 == 0 (Par?)	Ação em total	Valor de total
<b>Início</b>	-	-	-	0
1	1 ≤ 5 (V)	1 (mod 2) = 1 (Falso)	Nenhuma	0
2	2 ≤ 5 (V)	2 (mod 2) = 0 (Verdadeiro)	$total \leftarrow 0 + 2$	2
3	3 ≤ 5 (V)	3 (mod 2) = 1 (Falso)	Nenhuma	2

Valor de <code>i</code>	Condição ( <code>i &lt;= 5</code> )	<code>i mod 2 == 0</code> (Par?)	Ação em <code>total</code>	Valor de <code>total</code>
4	$4 \leq 5$ (V)	$4 \bmod 2 = 0$ (Verdadeiro)	<code>total</code> $\leftarrow 2 + 4$	6
5	$5 \leq 5$ (V)	$5 \bmod 2 = 1$ (Falso)	Nenhuma	6
6	$6 \leq 5$ (F)	Encerra o laço	-	<b>Saída: 6</b>

#### Exercício 4: Laços de Repetição Aninhados

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 1; i <= 3; i++):
 para (j ← 1; j <= i; j++):
 total ← total + j
imprimir(total)
```

#### Resolução e Tabela de Rastreio:

Loop Externo (i)	Loop Interno (j)	Condição (j <= i)	Cálculo ( <code>total + j</code> )	Novo <code>total</code>
-	-	-	Início	0
<b>i = 1</b>	j = 1	$1 \leq 1$ (V)	$0 + 1$	1
	j = 2	$2 \leq 1$ (F)	Sai do loop interno	1
<b>i = 2</b>	j = 1	$1 \leq 2$ (V)	$1 + 1$	2
	j = 2	$2 \leq 2$ (V)	$2 + 2$	4
	j = 3	$3 \leq 2$ (F)	Sai do loop interno	4
<b>i = 3</b>	j = 1	$1 \leq 3$ (V)	$4 + 1$	5
	j = 2	$2 \leq 3$ (V)	$5 + 2$	7
	j = 3	$3 \leq 3$ (V)	$7 + 3$	10
	j = 4	$4 \leq 3$ (F)	Sai do loop interno	10
<b>i = 4</b>	-	$4 \leq 3$ (F)	Encerra o programa	<b>Saída: 10</b>

### Módulo 3: Tipos de Erros em Programação

Identificar o tipo de erro é o primeiro passo para corrigi-lo com eficiência.

Tipo de Erro	Definição	Quando Ocorre?	Exemplo
<b>Erro de Sintaxe</b>	Violação das regras gramaticais da linguagem.		

| Durante a **compilação** (a IDE aponta a linha exata).

| Esqueceu um parêntese ou ponto e vírgula: `if (carro.isDisponivel() {.`

| | **Erro de Execução** (*Runtime*) | O código compila, mas quebra durante o processamento lançando uma exceção.

| Durante a **execução**.

| Acessar um ponteiro nulo (`NullPointerException`) ou divisão por zero.

| | **Erro de Lógica** | O programa roda até o fim sem quebrar, mas produz resultados incorretos.

| Durante a **execução** (difícil detecção, sem alerta do sistema).

| Calcular desconto aplicável apenas para `dias < 10` quando a regra era `dias >= 10`.

|

Outros tipos secundários incluem erros aritméticos, erros de recursos e erros semânticos.

---

## Módulo 4: Depuração de Código (*Debugging*)

Depurar é o processo de encontrar e reduzir falhas no código.

### Atalhos Fundamentais do Depurador (IntelliJ IDEA)

- **Breakpoint:** Marcação visual na linha do código onde a execução deve ser pausada temporariamente para análise.
- **Step Over (F8):** Executa a linha atual por completo e avança para a próxima instrução sem entrar em métodos.
- **Step Into (F7):** Entra no método sendo chamado na linha atual para depurar seu interior.
- **Step Out (Shift + F8):** Conclui a execução do método atual e retorna ao ponto onde ele foi chamado.

### Recursos Avançados de Depuração

- **Evaluate Expression:** Permite testar trechos de código, operações matemáticas ou inspecionar o estado das variáveis em tempo real sem alterar a execução do programa original.
- **Breakpoint Condicional:** O depurador interrompe a execução apenas se uma condição específica for verdadeira (ex.: `i == 50` dentro de um loop de 1000 repetições).
- **Técnica do Pato de Borracha (*Rubber Duck Debugging*):** Explicar o código linha por linha em voz alta para identificar divergências na lógica.

---

## Módulo 5: Documentação de Código e Javadoc

A documentação adequada previne perda de tempo, reduz bugs, facilita a manutenção em equipe e auxilia no uso correto de ferramentas de inteligência artificial.

### Níveis de Documentação

- **Baixo Nível:** Comentários no código-fonte, descrições de métodos e classes.
- **Alto Nível:** Diagramas UML, manuais do sistema e documentação arquitetural.

## Boas Práticas: Documentação Útil vs. Inútil

- **Cultura de Código Limpo:** Nomes claros de variáveis e métodos dispensam comentários óbvios.
- **Inútil:** Comentar o que o código já deixa evidente (ex.: `int x = 10; // atribui 10 para x`).
- **Útil:** Explicar o **motivo** de uma regra de negócio ou o comportamento de uma função.

## Padrão Javadoc (Java)

O **Javadoc** é a ferramenta oficial do Java para gerar documentação em formato HTML a partir de comentários estruturados. Começa com `/**` e termina com `*/`.

### Exemplo Prático em Java:

```
/**
 * Classe responsável por realizar operações financeiras de aluguel.
 *
 * @author Equipe de Desenvolvimento
 * @version 1.0
 */
public class CalculadoraAluguel {

 /**
 * Calcula o valor total do aluguel com base nos dias e na diária.
 *
 * @param valorDiaria O valor de uma diária em reais.
 * @param dias Quantidade de dias contratados.
 * @return O valor total calculado com desconto, se aplicável.
 */
 public double calcularValorTotal(double valorDiaria, int dias) {
 double total = valorDiaria * dias;
 if (dias >= 10) {
 total = total * 0.9; // 10% de desconto para 10 dias ou mais
 }
 return total;
 }
}
```

Para gerar a documentação em HTML no IntelliJ IDEA, acesse: Tools > Generate Javadoc.

```
content = """"# Material de Revisão: Análise e Projeto de Algoritmos
Depuração, Testes de Mesa, Tipos de Erros e Documentação de Código
```

```

```

### ## 1. Fundamentos e Representação de Algoritmos

Um **algoritmo** é uma sequência lógica, finita e bem definida de instruções para resolver um problema.

#### ### Estruturas Básicas de um Algoritmo

Todo algoritmo é construído combinando três estruturas fundamentais:

1. **Sequência:** Execução linear de instruções passo a passo, de cima para baixo.
2. **Seleção (Condiçõais):** Tomada de decisões com base em condições booleanas (`se / senão` ou

3. **\*\*Repetição (Laços/Loops):\*\*** Execução repetida de um bloco de código enquanto ou para determina

### Formas de Representação

\* **\*\*Pseudocódigo:\*\*** Descrição estruturada do algoritmo em linguagem simples próxima da linguagem h

\* **\*\*Fluxograma:\*\*** Representação gráfica e visual do fluxo de execução por meio de formas geométric

---

## 2. Teste de Mesa (Dry Run)

O **\*\*teste de mesa\*\*** é uma técnica manual para simular a execução de um algoritmo usando papel e ca

---

### Exercício 1: Condicional e Reatribuição de Variáveis

Dado o pseudocódigo abaixo:

```
```text
inicio
    a, b, c, d : inteiro;
    a ← 1;
    b ← 3;
    se (a + b >= 5) então:
        c ← 10;
    senão:
        c ← 30;
    fimse;
    d ← a + b;
    a ← c;
    b ← d;
    escreva(a, b, c, d);
fim
```

Resolução Passo a Passo (Tabela do Teste de Mesa):

Passo	Linha / Instrução	a	b	c	d	Condição (a + b >= 5)	Observação
1	a ← 1; b ← 3;	1	3	-	-	-	Inicialização de variáveis
2	se (a + b >= 5)	1	3	-	-	1 + 3 = 4 >= 5 (Falso)	Desvia para o bloco senão
3	c ← 30;	1	3	30	-	-	Atribuição dentro do senão
4	d ← a + b;	1	3	30	4	-	d = 1 + 3 = 4

Passo	Linha / Instrução	a	b	c	d	Condição (a + b >= 5)	Observação
5	a ← c;	30	3	30	4	-	a recebe o valor de c (30)
6	b ← d;	30	4	30	4	-	b recebe o valor de d (4)
7	escreva(...)	30	4	30	4	-	Impressão final

Resultado impresso: 30, 4, 30, 4

Exercício 2: Laço Acumulador (para)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 0; i < 3; i++):
    total ← total + i + 1
imprimir(total)
```

Resolução Passo a Passo:

Iteração	i	Condição (i < 3)	Operação (total + i + 1)	Valor Atualizado de total
Início	-	-	-	0
1ª	0	0 < 3 (V)	0 + 0 + 1 = 1	1
2ª	1	1 < 3 (V)	1 + 1 + 1 = 3	3
3ª	2	2 < 3 (V)	3 + 2 + 1 = 6	6
Fim	3	3 < 3 (F)	Laço encerrado	Saída: 6

Exercício 3: Laço com Filtro Condicional (mod)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 1; i <= 5; i++):
    se (i mod 2 == 0) então
        total ← total + i
imprimir(total)
```

Resolução Passo a Passo:

Passo	i	Condição (i <= 5)	i mod 2 == 0 (Par?)	Ação em total	Valor de total
1	-	-	-	Inicialização	0
2	1	Verdadeiro	1 mod 2 = 1 (Falso)	Não altera	0
3	2	Verdadeiro	2 mod 2 = 0 (Verdadeiro)	total ← 0 + 2	2
4	3	Verdadeiro	3 mod 2 = 1 (Falso)	Não altera	2
5	4	Verdadeiro	4 mod 2 = 0 (Verdadeiro)	total ← 2 + 4	6
6	5	Verdadeiro	5 mod 2 = 1 (Falso)	Não altera	6
7	6	Falso	-	Encerra laço	Saída: 6

Exercício 4: Laços Aninhados (Nested Loops)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 1; i <= 3; i++):
    para (j ← 1; j <= i; j++):
        total ← total + j
imprimir(total)
```

Resolução Passo a Passo:

Laço Externo (i)	Laço Interno (j)	Condição (j <= i)	Cálculo (total + j)	Novo total
-	-	-	Estado Inicial	0
i = 1	j = 1	1 ≤ 1 (V)	0 + 1	1
	j = 2	2 ≤ 1 (F)	Sai do laço interno	1
i = 2	j = 1	1 ≤ 2 (V)	1 + 1	2
	j = 2	2 ≤ 2 (V)	2 + 2	4
	j = 3	3 ≤ 2 (F)	Sai do laço interno	4
i = 3	j = 1	1 ≤ 3 (V)	4 + 1	5
	j = 2	2 ≤ 3 (V)	5 + 2	7
	j = 3	3 ≤ 3 (V)	7 + 3	10
	j = 4	4 ≤ 3 (F)	Sai do laço interno	10
i = 4	-	4 ≤ 3 (F)	Encerra programa	Saída: 10

3. Tipos de Erros em Programação

Tipo de Erro	Quando Ocorre?	Características	Exemplo
Erro de Sintaxe	Antes / Compilação	Violação das regras gramaticais da linguagem. A IDE/Compilador bloqueia a execução.	Esquecer ponto e vírgula ;, parênteses desalinhados ou palavras reservadas incorretas.
Erro de Execução (Runtime)	Durante a Execução	O código compila, mas aborta abruptamente lançando uma exceção durante o uso.	Divisão por zero (<code>ArithmeticException</code>), ponteiro nulo (<code>NullPointerException</code>) ou estouro de vetor (<code>ArrayIndexOutOfBoundsException</code>)
Erro de Lógica	Durante a Execução	O programa executa normalmente do início ao fim sem quebrar, porém gera um resultado incorreto .	Aplicar desconto apenas para <code>dias < 10</code> em vez de <code>dias >= 10</code> .

4. Depuração de Código (Debugging)

Depurar é o processo sistemático de localizar, analisar e corrigir falhas ou comportamentos indesejados no software.

Principais Comandos do Depurador (Ex: IntelliJ IDEA)

[Linha Atual do Breakpoint]

Step Over (F8)	Executa a linha atual sem entrar em métodos.
Step Into (F7)	Entra no método chamado na linha atual.
Step Out (Shift+F8)	Finaliza o método atual e retorna ao chamador.

Recursos Importantes:

- **Breakpoint:** Ponto de parada configurado pelo desenvolvedor para suspender a execução do programa em determinada linha.
- **Conditional Breakpoint:** Breakpoint acionado apenas quando uma expressão booleana for satisfeita (ex: `i == 50`).
- **Evaluate Expression (Alt + F8):** Janela interativa para testar expressões, operações matemáticas ou inspecionar objetos durante a pausa sem alterar o código original.
- **Rubber Duck Debugging (Técnica do Pato de Borracha):** Método de depuração conceitual que consiste em explicar a lógica do código linha por linha para um objeto inanimado ou colega.

5. Documentação de Código e Javadoc

Documentar o código reduz o tempo de manutenção, evita regressões de bugs, facilita a integração de novos membros da equipe e otimiza o uso de assistentes de inteligência artificial.

Boas Práticas: Documentação Útil vs. Inútil

- **Evite o óbvio:** Não reescreva o código em português (ex: `x++; // incrementa x`). Use nomes limpos e expressivos para métodos e variáveis.
- **Documente intenções:** Explique o **porquê** de regras de negócio, condições, pós-condições e limitações.

Padrão Javadoc (Java)

Formato de documentação oficial da linguagem Java. Os blocos iniciam por `/**` e terminam por `*/`.

```
/**
 * Realiza o cálculo do valor final de uma locação de veículo.
 *
 * @author Equipe de Desenvolvimento
 * @version 1.0
 */
public class ProcessadorDeLocacao {

    /**
     * Calcula o custo total com base na diária e no número de dias.
     * Aplica 10% de desconto para locações de 10 dias ou mais.
     *
     * @param valorDiaria O valor individual por dia contratado (deve ser maior que zero).
     * @param dias A quantidade total de dias de locação.
     * @return O valor total final com desconto aplicado se elegível.
     * @throws IllegalArgumentException se valorDiaria ou dias forem menores ou iguais a zero.
     */
    public double calcularTotal(double valorDiaria, int dias) {
        if (valorDiaria <= 0 || dias <= 0) {
            throw new IllegalArgumentException("Valor da diária e dias devem ser positivos.");
        }

        double total = valorDiaria * dias;
        if (dias >= 10) {
            total *= 0.90; // 10% de desconto
        }
        return total;
    }
}
```

Gerando Javadoc no IntelliJ: Menu Tools > Generate JavaDoc....

6. Checklist de Preparação para Avaliações

- ☒ Sei montar a tabela completa de um Teste de Mesa?
- ☒ Consigo diferenciar Erros de Sintaxe, Execução e Lógica?
- ☒ Conheço as teclas de atalho de depuração no IntelliJ (F7, F8, Shift+F8)?
- ☒ Sei para que serve um Breakpoint Condicional e a ferramenta Evaluate Expression?
- ☒ Sei aplicar as tags Javadoc (`@param`, `@return`, `@throws`, `@author`, `@version`)? “““

with open("revisao_algoritmos_depuracao.md", "w", encoding="utf-8") as f: f.write(content)

```
print("Arquivo revisao_algoritmos_depuracao.md criado com sucesso!")
```

```
```text?code_stdout&code_event_index=1
Arquivo revisao_algoritmos_depuracao.md criado com sucesso!
```

Sim, o arquivo em Markdown foi gerado e estruturado com todo o conteúdo de revisão, incluindo tabelas completas para os testes de mesa, explicações sobre os tipos de erro, atalhos do depurador no IntelliJ e o padrão Javadoc.

O arquivo `revisao_algoritmos_depuracao.md` foi criado. Segue o conteúdo completo do arquivo:

```
Material de Revisão: Análise e Projeto de Algoritmos
Depuração, Testes de Mesa, Tipos de Erros e Documentação de Código
```

```

```

## ## 1. Fundamentos e Representação de Algoritmos

Um **algoritmo** é uma sequência lógica, finita e bem definida de instruções para resolver um problema.

### ### Estruturas Básicas de um Algoritmo

Todo algoritmo é construído combinando três estruturas fundamentais:

1. **Sequência**: Execução linear de instruções passo a passo, de cima para baixo.
2. **Seleção (Condicional)**: Tomada de decisões com base em condições booleanas (`se / senão` ou `if / else`).
3. **Repetição (Laços/Loops)**: Execução repetida de um bloco de código enquanto ou para determinado número de vezes.

### ### Formas de Representação

- \* **Pseudocódigo**: Descrição estruturada do algoritmo em linguagem simples próxima da linguagem humana.
- \* **Fluxograma**: Representação gráfica e visual do fluxo de execução por meio de formas geométricas.

```

```

## ## 2. Teste de Mesa (Dry Run)

O **teste de mesa** é uma técnica manual para simular a execução de um algoritmo usando papel e caneta.

```

```

### ### Exercício 1: Condicional e Reatribuição de Variáveis

Dado o pseudocódigo abaixo:

```
```text
inicio
    a, b, c, d : inteiro;
    a ← 1;
    b ← 3;
    se (a + b >= 5) então:
        c ← 10;
    senão:
        c ← 30;
    fimse;
```

```

    d ← a + b;
    a ← c;
    b ← d;
    escreva(a, b, c, d);
fim

```

Resolução Passo a Passo (Tabela do Teste de Mesa):

Passo	Linha / Instrução	a	b	c	d	Condição (a + b >= 5)	Observação
1	a ← 1; b ← 3;	1	3	-	-	-	Inicialização de variáveis
2	se (a + b >= 5)	1	3	-	-	1 + 3 = 4 >= 5 (Falso)	Desvia para o bloco senão
3	c ← 30;	1	3	30	-	-	Atribuição dentro do senão
4	d ← a + b;	1	3	30	4	-	d = 1 + 3 = 4
5	a ← c;	30	3	30	4	-	a recebe o valor de c (30)
6	b ← d;	30	4	30	4	-	b recebe o valor de d (4)
7	escreva(...)	30	4	30	4	-	Impressão final

Resultado impresso: 30, 4, 30, 4

Exercício 2: Laço Acumulador (para)

Dado o pseudocódigo abaixo:

```

total ← 0
para (i ← 0; i < 3; i++):
    total ← total + i + 1
imprimir(total)

```

Resolução Passo a Passo:

Iteração	i	Condição (i < 3)	Operação (total + i + 1)	Valor Atualizado de total
Início	-	-	-	0

Iteração	i	Condição (i < 3)	Operação (total + i + 1)	Valor Atualizado de total
1ª	0	0 < 3 (V)	0 + 0 + 1 = 1	1
2ª	1	1 < 3 (V)	1 + 1 + 1 = 3	3
3ª	2	2 < 3 (V)	3 + 2 + 1 = 6	6
Fim	3	3 < 3 (F)	Laço encerrado	Saída: 6

Exercício 3: Laço com Filtro Condicional (mod)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 1; i <= 5; i++):
    se (i mod 2 == 0) então
        total ← total + i
imprimir(total)
```

Resolução Passo a Passo:

Passo	i	Condição (i <= 5)	i mod 2 == 0 (Par?)	Ação em total	Valor de total
1	-	-	-	Inicialização	0
2	1	Verdadeiro	1 mod 2 = 1 (Falso)	Não altera	0
3	2	Verdadeiro	2 mod 2 = 0 (Verdadeiro)	total ← 0 + 2	2
4	3	Verdadeiro	3 mod 2 = 1 (Falso)	Não altera	2
5	4	Verdadeiro	4 mod 2 = 0 (Verdadeiro)	total ← 2 + 4	6
6	5	Verdadeiro	5 mod 2 = 1 (Falso)	Não altera	6
7	6	Falso	-	Encerra laço	Saída: 6

Exercício 4: Laços Aninhados (Nested Loops)

Dado o pseudocódigo abaixo:

```
total ← 0
para (i ← 1; i <= 3; i++):
    para (j ← 1; j <= i; j++):
        total ← total + j
imprimir(total)
```

Resolução Passo a Passo:

Laço Externo (i)	Laço Interno (j)	Condição (j <= i)	Cálculo (total + j)	Novo total
-	-	-	Estado Inicial	0
i = 1	j = 1	$1 \leq 1$ (V)	0 + 1	1
	j = 2	$2 \leq 1$ (F)	Sai do laço interno	1
i = 2	j = 1	$1 \leq 2$ (V)	1 + 1	2
	j = 2	$2 \leq 2$ (V)	2 + 2	4
	j = 3	$3 \leq 2$ (F)	Sai do laço interno	4
i = 3	j = 1	$1 \leq 3$ (V)	4 + 1	5
	j = 2	$2 \leq 3$ (V)	5 + 2	7
	j = 3	$3 \leq 3$ (V)	7 + 3	10
	j = 4	$4 \leq 3$ (F)	Sai do laço interno	10
i = 4	-	$4 \leq 3$ (F)	Encerra programa	Saída: 10

3. Tipos de Erros em Programação

Tipo de Erro	Quando Ocorre?	Características	Exemplo
Erro de Sintaxe	Antes / Compilação	Violação das regras gramaticais da linguagem. A IDE/Compilador bloqueia a execução.	Esquecer ponto e vírgula ;, parênteses desalinhados ou palavras reservadas incorretas.
Erro de Execução (Runtime)	Durante a Execução	O código compila, mas aborta abruptamente lançando uma exceção durante o uso.	Divisão por zero (ArithmeticException), ponteiro nulo (NullPointerException) ou estouro de vetor (ArrayIndexOutOfBoundsException)
Erro de Lógica	Durante a Execução	O programa executa normalmente do início ao fim sem quebrar, porém gera um resultado incorreto .	Aplicar desconto apenas para dias < 10 em vez de dias >= 10.

4. Depuração de Código (Debugging)

Depurar é o processo sistemático de localizar, analisar e corrigir falhas ou comportamentos indesejados no software.

Principais Comandos do Depurador (Ex: IntelliJ IDEA)

[Linha Atual do Breakpoint]

Step Over (F8)	Executa a linha atual sem entrar em métodos.
Step Into (F7)	Entra no método chamado na linha atual.

Step Out (Shift+F8) Finaliza o método atual e retorna ao chamador.

Recursos Importantes:

- **Breakpoint:** Ponto de parada configurado pelo desenvolvedor para suspender a execução do programa em determinada linha.
 - **Conditional Breakpoint:** Breakpoint acionado apenas quando uma expressão booleana for satisfeita (ex: `i == 50`).
 - **Evaluate Expression (Alt + F8):** Janela interativa para testar expressões, operações matemáticas ou inspecionar objetos durante a pausa sem alterar o código original.
 - **Rubber Duck Debugging (Técnica do Pato de Borracha):** Método de depuração conceitual que consiste em explicar a lógica do código linha por linha para um objeto inanimado ou colega.
-

5. Documentação de Código e Javadoc

Documentar o código reduz o tempo de manutenção, evita regressões de bugs, facilita a integração de novos membros da equipe e otimiza o uso de assistentes de inteligência artificial.

Boas Práticas: Documentação Útil vs. Inútil

- **Evite o óbvio:** Não reescreva o código em português (ex: `x++; // incrementa x`). Use nomes limpos e expressivos para métodos e variáveis.
- **Documente intenções:** Explique o **porquê** de regras de negócio, precondições, pós-condições e limitations.

Padrão Javadoc (Java)

Formato de documentação oficial da linguagem Java. Os blocos iniciam por `/**` e terminam por `*/`.

```
/**
 * Realiza o cálculo do valor final de uma locação de veículo.
 *
 * @author Equipe de Desenvolvimento
 * @version 1.0
 */
public class ProcessadorDeLocacao {

    /**
     * Calcula o custo total com base na diária e no número de dias.
     * Aplica 10% de desconto para locações de 10 dias ou mais.
     *
     * @param valorDiaria O valor individual por dia contratado (deve ser maior que zero).
     * @param dias A quantidade total de dias de locação.
     * @return O valor total final com desconto aplicado se elegível.
     * @throws IllegalArgumentException se valorDiaria ou dias forem menores ou iguais a zero.
     */
    public double calcularTotal(double valorDiaria, int dias) {
        if (valorDiaria <= 0 || dias <= 0) {
            throw new IllegalArgumentException("Valor da diária e dias devem ser positivos.");
        }
    }
}
```

```

    double total = valorDiaria * dias;
    if (dias >= 10) {
        total *= 0.90; // 10% de desconto
    }
    return total;
}
}

```

Gerando Javadoc no IntelliJ: Menu Tools > Generate Javadoc....

6. Checklist de Preparação para Avaliações

- ☒ Sei montar a tabela completa de um Teste de Mesa?
- ☒ Consigo diferenciar Erros de Sintaxe, Execução e Lógica?
- ☒ Conheço as teclas de atalho de depuração no IntelliJ (F7, F8, Shift+F8)?
- ☒ Sei para que serve um Breakpoint Condicional e a ferramenta Evaluate Expression?
- ☒ Sei aplicar as tags Javadoc (@param, @return, @throws, @author, @version)?

Deseja que eu envie/faça o upload deste arquivo ****md**** para o seu Google Drive?